A VISUAL TOUR · SYMBOL BY SYMBOL

NS

golf-terse · statically typed · native code via LLVM

```
1..11[!_%2]{^^2}/+ → 220
```

Sum of squares of the even numbers below 11. By the last slide, every symbol here reads like a word.

01 · HOW TO READ THIS DECK

Five colours, five jobs

```
42
"hi"
T F
```

LITERALS

numbers, strings, bools, ranges

```
+ - *
^^ %
= !
```

OPERATORS

maths, compare, logic

```
{ }
[ ]
/ \
```

BLOCKS

map, filter, fold, scan

```
_ @
$
_1 _2
```

IMPLICIT

names a block gets for free

```
f:x
x.f
sort
```

CALLS

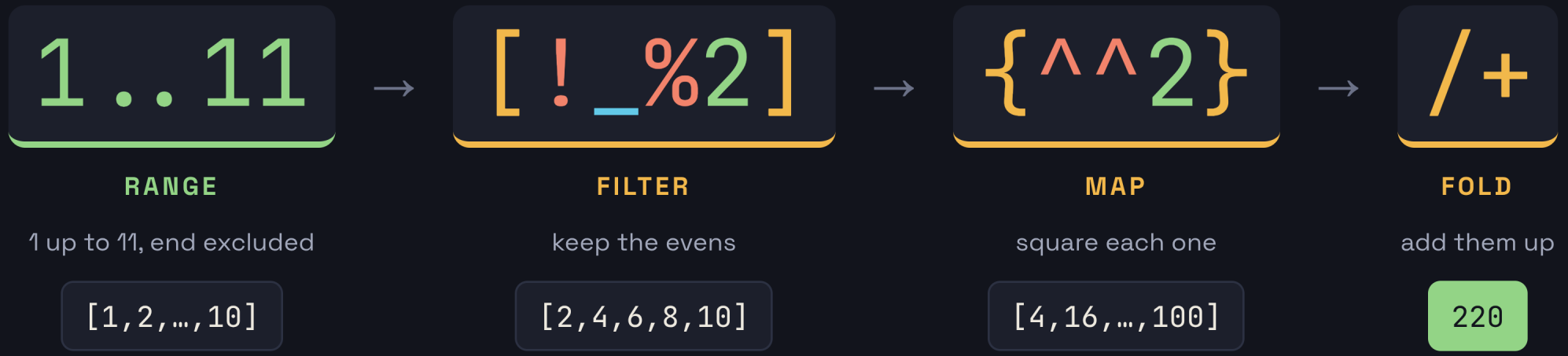
functions and how to call them



No type annotations anywhere — yet every expression has one fixed type at compile time.

02 · ANATOMY OF ONE LINE

Read it left to right, like a pipe



The compiler fuses the whole chain into **one loop** — no temporary arrays are built.

03 · ZOOM IN

Six glyphs, one filter

`[` **FILTER OPENS**

glued to a value, so not a list

`!` **NOT**

0 → T, anything else → F

`_` **ELEMENT**

the item being tested

`%` **MODULO**

remainder after dividing

`2` **LITERAL**

a plain int

`]` **FILTER CLOSES**

keep items where body is truthy

PRECEDENCE

```
!_%2  =  !(_%2)
```

! binds looser than maths and comparison, like Python's not

_	1	2	3	4	5
_%2	1	0	1	0	1
!_%2	F	T	F	T	F
kept	–	2	–	4	–

04 · CALLING FUNCTIONS

Colon calls, dot chains

```
println : 1+2*3
```

→ 7

NAME CALL ARGS → END OF STATEMENT

`add:3,4` commas separate arguments

```
[5,3,8].sort.println
= println:(sort:[5,3,8])
```

X.F IS F:X — PIPELINES READ LEFT TO RIGHT

⚠ GOTCHA

```
int:"42"+1
```

= int:("42"+1) → **421**

```
(int:"42")+1
```

parens stop the args → **43**

The left operand decides the type

`"1"`	`+1`	→	`"11"`	**STRING** 1 becomes ”1”, then concatenate
`1`	`+"1"`	→	`2`	**INT** ”1” parsed as 1, then add
`1`	`+2.7`	→	`3`	**INT** 2.7 truncated to 2
`1.0`	`+2`	→	`3.0`	**FLOAT** 2 widened to 2.0
`"ab"`	`*3`	→	`"ababab"`	**STRING** string × int repeats

No type errors between primitives: the right side is **converted to the left’s type.** Same for variables — the first assignment fixes the type.

06 · IMPLICIT NAMES

Blocks never declare parameters

_

CURRENT ELEMENT

```
1..6{_*_}
```

→ [1,4,9,16,25]

@

CURRENT INDEX

```
"abc"{"{@}{_}"}
```

→ [0a,1b,2c]

$

ACCUMULATOR

```
1..6/{$+_}
```

→ 15

_1_2

ARGUMENTS

```
add={_1+_2}
```

add:3,4 → 7

`{ _ { _ } }` Nested blocks: the **innermost** owns _ @ $. Save an outer one first, e.g. p=_.

07 · FOUR BRACKETS, FOUR LOOPS

Bracket shape = what happens

x{ } MAP

`1..6{_*_}` · transform each

1 2 3 4 5 → 1 4 9 16 25

x[ ] FILTER

`1..6[_%2]` · keep the truthy

1 ~~2~~ 3 ~~4~~ 5 → 1 3 5

x/{ } FOLD

`1..6/{$+_}` · squash to one

1 2 3 4 5 → 15 $ starts as the first item

x\{ } SCAN

`1..6\{$+_}` · every fold step

1 2 3 4 5 → 1 3 6 10 15

Glued to a value = an operator · [with a space or at the start = a list literal · / without { or an operator after it = divide

Start with an operator, and _ fills itself in

```
1..6{*10}      ≡ {_*10}      → [10,20,30,40,50]
1..10[>6]      ≡ [_>6]       → [7,8,9]
xs{^^2}        ≡ {_^^2}      operator section · ^^ is power
ws{int}        ≡ {int:_}     function section · a bare name is called on _
```

⚠ - never starts a section: `xs{-1}` is just the constant −1 for every item.

09 · OPERATOR FOLDS

Slash or backslash + one operator

glyph	means	[3,8,2] →
/+	sum	13
/*	product	48
/>	max	8
/<	min	2
/& /|	all / any	[T,F] → F / T
\+	running sum	[3,11,13]
*	running product	[3,24,48]

FACTORIALS IN 7 CHARS

```
1..11\*
```

→ [1,2,6,24,120,…]

FOLD OR DIVIDE?

`x/-` vs `x/-1`

fold only when nothing follows the operator

10 · ARRAYS, RANGES, INDEXING

@ points in, .. spans, # counts



top: index from the front

bottom: negative, from the end

amber: the slice xs@1..3

`xs@0` → 3
index

`xs@-1` → 5
last item

`xs@1..3` → [1,4]
slice, end excluded

`#xs` → 5
length

`0..n-1`
the end may be arithmetic

`0..xs`
array as end = its length

`[0]*4` → [0,0,0,0]
repeat

`xs++7`
append (or ++ array: concat)

11 · CONTROL FLOW

No if, no while — just ? and ~{

`c ? a : b`

IF / ELSE
a value, nests right

`c ? a`

IF
runs a only when c holds

`(s1; s2)`

GROUP
several statements, value = last

`c ~{ … }`

WHILE
c re-checked before each pass

`1..4{…}`

FOR EACH
a map is the for-loop

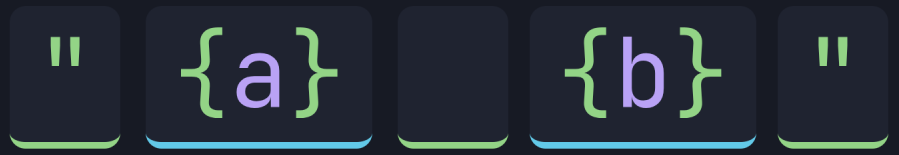
COLLATZ STEPS FOR 27

```
n = 27; c = 0
n>1 ~{
n = n%2 ? 3*n+1 : n/2
c += 1
}
println:c
```

→ 111

odd? 3n+1 : n/2 — one ternary, no if keyword

Braces inside quotes, letters as an array



→ "2 1"

escape a literal brace with \{

```
"hello"[!_="l"]
```

h e ~~l~~ ~~l~~ o → "heo"

filtering a string gives a string

```
#"hello" → 5
"hello"@?"ll" → T
ord:"a" → 97
"ab"*3 → ababab
"abc"@-1 → c
split:"a b"," "
```

13 · COLLECTIONS

A prefix in front of [picks the structure

→

HASH MAP

`["a"→1]`

empty: [->] · missing key reads 0

%[

HASH SET

`%[1,2,3]`

& | - are inter, union, diff

Q[

QUEUE

`Q[1,2]`

first in, first out

PQ[

MIN-HEAP

`PQ[(5,"e"),(1,"a")]`

pops smallest; tuples order by first item

`m@k` get	`m@k = v` set	`@?` contains	`#` size
`+=` add / push	`-=` remove	`.pop` take front / top	`.keys` map keys

Maps, sets, queues are **shared** on assignment · arrays and strings are **copied** · in a map block, `_` is the value and `@` the key

14 · SAME GLYPH, DIFFERENT JOB

Position decides what a symbol means

glyph	here…	means	there…	means
#	# note	comment (space after)	#xs	length
[	xs[>2]	filter (glued)	[1,2]	list literal
/	xs/+	fold	a/2	divide
%	%[1,2]	set literal	a%2	modulo
:	sq:x	call (sq is a function)	c?a:b	else
=	x = 1	assign (statement start)	a=b ? …	equal?
@	xs@0	index	{@}	current index
-	f:-1	negative number	a-1	minus
^	6^3	xor	2^^10	power

PART TWO

Now read real programs

```
println:1..1000[!_%3|!_%5]/+          Project Euler #1
2..50[p=_; !2..p[!p%_]]               primes
q = {… q:_[<p] …}                     quicksort
w{g@(_{_}.sort.join:"") ++= _}        anagram groups
q ~{ du,u = q.pop … }                 Dijkstra
```

15 · PROJECT EULER #1

Sum every multiple of 3 or 5 below 1000

```
println: 1..1000 [!_%3|!_%5] /+ 233168
```

CALL — everything after is the argument

RANGE — 1 … 999

FILTER — divisible by 3 **or** by 5

FOLD — sum

HOW IT GROUPS — TIGHTEST FIRST

```
println:(1..1000[(!(_%3)) | (!(_%5))]/+)
```

postfix `[ ] /+` > `* / %` > `+ -` > `= < >` > `!` > `& ^ |` > `? :` > f: call

16 · PRIMES · NESTED BLOCKS

A filter inside a filter

```
2..50[p=_; !2..p[!p%_]] → [2,3,5,7,11,…,47]
```

OUTER FILTER · _ = CANDIDATE

`p=_;` save it — the inner block will take over _

`!` empty list is falsy → ! makes it T → keep

INNER FILTER · _ = DIVISOR

`2..p[!p%_]` numbers in 2…p−1 that divide p

p	divisors	kept?
7	[]	yes
9	[3]	no
12	[2,3,4,6]	no
13	[]	yes

17 · QUICKSORT IN ONE LINE

A recursive block, three filters, two joins

```
q = {
```
define q

```
p = #_ ? _@0 : 0;
```
pivot = first item (0 if empty)

```
#_<2 ? _ :
```
0–1 items: already sorted

```
(q:_[<p])
```
sort the smaller

```
++_[=p]++
```
equal ones in the middle

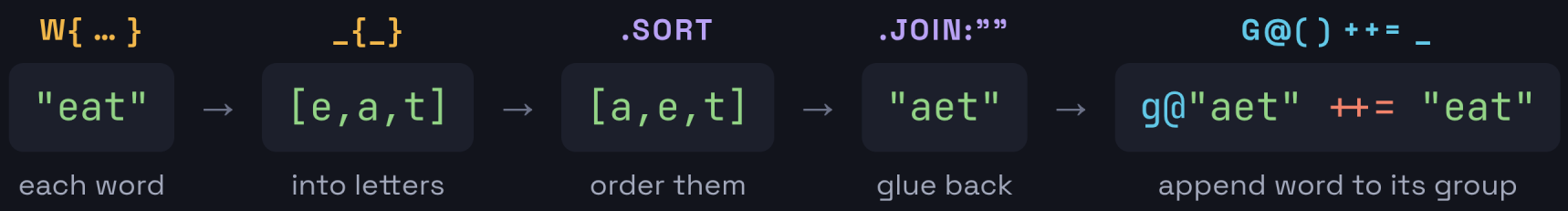
```
(q:_[>p])}
```
sort the bigger



18 · GROUP ANAGRAMS

Sorted letters become the map key

```
w{g@(_{_}.sort.join:"") ++= _}
```




```
aet eat tea ate
```

```
ant tan nat
```

```
abt bat
```

19 · DIJKSTRA'S CORE LOOP

Pop the nearest, relax its edges, repeat

```
q ~{
```
while queue isn't empty

```
du,u = q.pop
```
take nearest (dist, node), unpack it

```
e[ (_@0)=u & du+_@2<d@(_@1) ]{ v=_@1; d@v = du+_@2;
```
edges… …leaving u — and — …that shorten the way to their end — for each: — v = edge end — new shortest distance

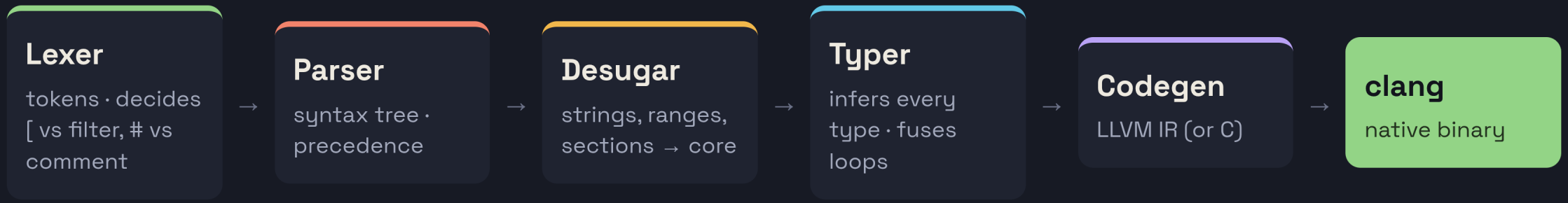
```
pr@v = u; q+=(d@v,v)}
```
remember the parent — push (dist, node)

```
}
```

```
d → [0,3,1,4,7]
```

```
path 0 → 2 → 1 → 3 → 4
```

From glyphs to a native binary



No annotations

every type inferred at compile time

Per call site

sq:7 and sq:1.5 compile separately

Fused chains

x[…]{…}/+ is one loop

Refcounted

no garbage collector

21 · CHEAT SHEET

Every glyph on one page

`x{…}` map	`x[…]` filter	`x/{…}` fold	`x\{…}` scan	`x/+` sum (op fold)	`x\*` running product
`_` element	`@` index / key	`$` accumulator	`_1 _2` arguments	`f:a,b` call	`x.f` = f:x
`a..b` range, b excluded	`xs@i` index, -1 = last	`xs@a..b` slice	`#x` length	`@?` contains	`++` concat / append
`c?a:b` if / else	`c ~{…}` while	`(s; s)` group	`^^` power (^ = xor)	`"{x}"` interpolate	`# …` comment
`[a→1]` map	`%[ ]` set	`Q[ ]` queue	`PQ[ ]` min-heap	`+= -=` add / remove	`.pop` take front / top